



Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências – Bauru
Departamento de Computação

Sistema Unespão

Plataforma de Pedidos Personalizados para Padarias

Engenharia de Software II

David Almeida Saragó – RA: 241021081

Guilherme Molina Trindade – RA: 241023998

Lucas Costa Oliveira – RA: 241023076

Thiago Mitsuo da Silva Nomura – RA: 241023319

Bauru – SP
2026

Sumário

1	Introdução e Objetivos	1
1.1	Contexto do software	1
1.2	Escopo	2
1.3	Visão geral dos requisitos	3
1.4	Objetivos de qualidade	5
2	Arquitetura do Sistema	7
2.1	Contexto e fronteiras do sistema	7
2.2	Estilo ou padrão arquitetural	9
2.3	Contêineres e componentes	10
2.4	Decisões arquiteturais relevantes	12
2.5	Restrições de arquitetura	12
3	Projeto de Componentes	14
3.1	Visão geral dos componentes	14
3.2	Detalhamento dos componentes principais	16
3.3	Princípios de projeto	18
3.4	Padrões de projeto	19
4	Projeto de Interface do Usuário	22
4.1	Estado atual dos artefatos de interface	22
4.2	Canais de interação do Cliente	22
4.3	Fluxo principal de interação do cliente	23
4.4	Decisão de projeto: autenticação no totem físico	24
4.5	Interface do Atendente/Administrador	24
5	Especificação e Estratégia de Testes	26
5.1	Planejamento e níveis de teste	26
5.2	Testes de unidade	27
5.3	Testes de integração	30
5.4	Critérios de conclusão e cobertura	31
5.5	Automação e ferramentas	31
6	Gestão de Configuração e Manutenção	33
6.1	Repositório e itens de configuração	33
6.2	Gestão de dependências e alterações	34
6.3	Rastreabilidade	34
6.4	<i>Build</i> , integração e entrega	35
	Controle de Versões do Documento	36

Capítulo 1

Introdução e Objetivos

O Sistema Unespão é uma plataforma digital para a padaria Unespão que permite a clientes montarem e finalizarem pedidos de lanches personalizados, no estilo adotado por redes como Subway e Spoleto. Este capítulo situa o problema que o software resolve, delimita seu escopo, apresenta uma visão geral dos requisitos identificados a partir do domínio do negócio e estabelece os objetivos de qualidade que devem orientar as decisões de projeto detalhadas nos capítulos seguintes.

1.1 Contexto do software

A padaria Unespão opera sob um modelo de atendimento em que o cliente não escolhe apenas entre itens de cardápio fixos, mas monta o próprio lanche a partir de uma base (por exemplo, tipo de pão, massa de bolo ou base de salada) combinada com ingredientes adicionais (recheios, molhos, coberturas e acompanhamentos). Esse processo de personalização, hoje realizado sem suporte informatizado dedicado, sofre com a lentidão no atendimento, com a dificuldade de manter o controle de estoque dos insumos disponíveis e com a ausência de mecanismos que aproveitem o histórico de consumo do cliente para oferecer recomendações relevantes.

A finalidade do sistema, conforme definida pela equipe, é otimizar o processo de pedido e personalização de produtos, melhorar a experiência do cliente e aumentar a fidelização por meio de sugestões personalizadas de lanches. Para isso, o sistema passa a intermediar toda a jornada do cliente: da escolha da base e dos ingredientes até o registro do pedido, o pagamento da transação, a avaliação do prato consumido e o recebimento de sugestões baseadas em seu histórico.

Dois pontos dessa jornada aparecem com mais destaque na Visão Arquitetural (C4 Nível 1 e Nível 2) do que no relato inicial de Minimundo, e por isso merecem uma menção explícita aqui. O primeiro é o pagamento: o sistema se integra a um gateway de pagamento externo para processar as transações financeiras dos pedidos, e o **PedidoService** responde pelo ciclo do pedido desde a criação até a personalização dos itens e o processamento do pagamento. Do ponto de vista do cliente da padaria, um pedido só está de fato concluído depois de pago; por isso tratamos pagamento como parte natural do fluxo descrito neste capítulo, não como uma etapa à parte. O segundo ponto é a autenticação: o cliente se identifica junto à plataforma por meio de um serviço de autenticação externo (Google, via OAuth 2.0), o que permite ao sistema reconhecer um cliente recorrente, associar seu histórico de pedidos e, a partir daí, sustentar a geração de sugestões personalizadas.

O principal interessado direto no sistema é o cliente, que interage com a plataforma por meio de totens físicos instalados na loja ou de um aplicativo móvel, ambos servindo como canal de montagem, pagamento e envio dos pedidos. A padaria, como operadora do negócio, é a parte interessada institucional: é ela quem se beneficia do aumento de fidelização e da otimização

operacional que o sistema busca proporcionar. Quem de fato mantém o catálogo de produtos e ingredientes e cuida do controle de estoque no dia a dia, porém, não é a “padaria” em abstrato, e sim um segundo ator humano, o Atendente/Administrador, com um perfil de acesso mais simples do que o do cliente e voltado às tarefas operacionais de retaguarda. Ele não monta nem paga pedidos, nem gerencia contas de cliente; sua função no sistema é cadastrar e atualizar produtos base, ingredientes e níveis de estoque.

1.2 Escopo

Com base no minimundo e nos conceitos-chave do domínio descritos pela equipe, o Sistema Unespão compreende as seguintes capacidades de negócio:

- **Cadastro e gestão de clientes**, realizado de forma autônoma pelo próprio cliente por meio da autenticação com um provedor de identidade externo (Google, OAuth 2.0), sem intervenção do Atendente/Administrador.
- **Gestão de catálogo e estoque** (produtos base, ingredientes e respectivos níveis de estoque), mantendo o catálogo e a disponibilidade dos insumos – capacidade operada pelo ator Atendente/Administrador.
- **Montagem, registro e gerenciamento de pedidos**, isto é, a combinação de um produto base com ingredientes selecionados em um ou mais itens personalizados, agrupados em um pedido, incluindo a possibilidade de o cliente cancelar ou editar um item do pedido antes da confirmação do pagamento.
- **Processamento de pagamento do pedido**, por meio de integração com um gateway de pagamento externo, e **autenticação do cliente**, por meio de um provedor de identidade externo (Google, OAuth 2.0), ambos como parte do ciclo de vida do pedido.
- **Avaliação de prato**, permitindo que o cliente forneça nota e comentários sobre um item personalizado após a conclusão do pedido.
- **Geração de sugestões personalizadas** de lanches para clientes recorrentes, com base no histórico de pedidos e nas avaliações registradas.
- **Interação do cliente com o sistema** por meio de totem físico ou aplicativo móvel.

Esses pontos correspondem aos conceitos centrais do domínio descritos no Minimundo (Cliente, Produto Base, Ingrediente, Pedido, Item Personalizado, Avaliação de Prato, Sugestão Personalizada, Totem/Aplicativo, Histórico de Pedidos e Gerenciamento de Estoque), complementados pelo pagamento e pela autenticação, que tratamos aqui como parte inseparável, do ponto de vista do cliente, da ação de montar e finalizar um pedido.

A padaria Unespão modelada neste projeto é uma loja única. A menção a uma API de Localização/CEP na Visão de Contexto C4 (Nível 1), voltada a “identificar a unidade mais próxima” do cliente, é um recurso pensado para uma eventual expansão futura do produto a múltiplas unidades, e não algo implementado no escopo atual. Por isso, o sistema aqui descrito não trata de rede de lojas nem de seleção de unidade.

Fora do escopo

Ficam fora do escopo do Sistema Unespão:

- **Logística de entrega:** o modelo de negócio é de consumo dentro da própria loja, via totem ou aplicativo, sem serviço de delivery.
- **Gestão financeira e contábil da padaria:** o sistema processa o pagamento pontual de cada pedido através do gateway externo, mas não cobre fluxo de caixa, contabilidade ou relatórios financeiros do negócio.
- **Gestão de funcionários e recursos humanos:** folha de pagamento, escalas de trabalho e demais processos de RH não são tratados pelo sistema e não aparecem em nenhuma das fontes do projeto.
- **Atendimento presencial paralelo ao digital:** esse é um aspecto da operação humana da padaria, e não uma funcionalidade a ser suportada pelo software.

1.3 Visão geral dos requisitos

A partir do minimundo, dos conceitos-chave do domínio e da visão de contexto/containers da arquitetura, dá para inferir com segurança o seguinte conjunto de capacidades funcionais que o sistema deve oferecer:

Tabela 1.1: Requisitos funcionais inferidos das fontes do projeto.

Requisito funcional (inferido)	Descrição
Cadastro e consulta de clientes	Registrar informações básicas do cliente e associar seu histórico de pedidos.
Autenticação do cliente	Validar a identidade do cliente por meio de um provedor de identidade externo (Google, OAuth 2.0) antes de permitir a montagem e o envio de pedidos.
Gestão de catálogo e estoque pelo Atendente/Administrador	Permitir que o ator Atendente/Administrador cadastre, edite e mantenha produtos base, ingredientes e os respectivos níveis de estoque.
Montagem de item personalizado	Permitir a combinação de um produto base com um ou mais ingredientes, gerando um item personalizado único.
Realização de pedido	Registrar uma transação de cliente contendo um ou mais itens personalizados.
Cancelamento ou edição de item do pedido	Permitir que o cliente remova ou altere um item já incluído no pedido, desde que isso ocorra antes da confirmação do pagamento.
Processamento de pagamento	Processar o pagamento do pedido por meio de um gateway de pagamento externo, como etapa de conclusão da transação.
Avaliação de prato	Coletar nota e comentários do cliente sobre um item personalizado, após a conclusão do pedido.
Geração de sugestões personalizadas	Propor novos itens personalizados a clientes recorrentes, com base em histórico de pedidos e avaliações.
Consulta de histórico de pedidos	Disponibilizar o registro detalhado dos pedidos anteriores de um cliente.
Gerenciamento de estoque	Controlar e monitorar a quantidade disponível de produtos base e ingredientes.
Interação via totem/aplicativo	Oferecer interface digital (totem físico ou app móvel) para o cliente montar, pagar e enviar pedidos.

Quanto aos requisitos não funcionais, cabe registrar que as fontes utilizadas neste capítulo não constituem uma especificação formal de requisitos, de modo que os atributos de qualidade não aparecem detalhados de forma exaustiva. Ainda assim, dá para observar três indícios textuais relevantes, desenvolvidos na próxima seção: a finalidade do sistema enfatiza “melhorar a experiência do cliente” (usabilidade); o Minimundo destaca a necessidade de “garantir a disponibilidade dos itens” por meio do Gerenciamento de Estoque (confiabilidade/disponibilidade de dados); e a Visão Arquitetural descreve explicitamente uma preocupação de segurança ao isolar tokens de autenticação e dados de pagamento no backend, “fora do navegador” (segurança dos dados e das transações do cliente).

Sobre retenção de dados, optamos por manter o histórico de pedidos de um cliente enquanto sua conta permanecer ativa na plataforma, sem definir um prazo de expiração. Não há, nas fontes do projeto, indicação de uma política de retenção mais restritiva, e um histórico persistente é justamente o que sustenta a geração de sugestões personalizadas. Já metas quantitativas de desempenho em horário de pico e requisitos formais de portabilidade entre totem e aplicativo não são detalhados neste trabalho: tratamos esses pontos como não especificados no escopo acadêmico deste projeto, o que é compatível com o nível de detalhamento pedido para este

documento.

1.4 Objetivos de qualidade

Quatro objetivos de qualidade emergem com segurança do texto de Introdução, do Minimundo e da Visão Arquitetural, por estarem diretamente ligados à finalidade declarada do sistema:

Tabela 1.2: Objetivos de qualidade do Sistema Unespão.

Objetivo	Prioridade	Impacto esperado no projeto
Usabilidade / Experiência do cliente	Alta	A finalidade do sistema é explicitamente “melhorar a experiência do cliente” na personalização e no pedido de produtos; isso deve orientar o projeto das interfaces de totem e aplicativo para que a montagem de um item personalizado, o pagamento e o envio do pedido sejam simples e rápidos.
Confiabilidade da informação de estoque	Alta	O Minimundo indica que o sistema “considera o Gerenciamento de Estoque de Produtos Base e Ingredientes” para “garantir a disponibilidade dos itens”; isso implica que as informações de estoque exibidas ao cliente e usadas na montagem de itens personalizados precisam refletir corretamente a disponibilidade real, evitando pedidos de itens indisponíveis.
Segurança dos dados e transações do cliente	Alta	O sistema integra um provedor de autenticação externo (Google OAuth 2.0) e um gateway de pagamento para processar as transações financeiras dos pedidos; a própria Visão Arquitetural registra a preocupação de manter tokens de autenticação fora do navegador. Isso implica que credenciais do cliente e dados de pagamento não devem ser armazenados ou expostos indevidamente pelo sistema, delegando ao gateway externo o tratamento de informações financeiras sensíveis.
Relevância das sugestões personalizadas	Média	A finalidade do sistema também menciona “aumentar a fidelização através de sugestões personalizadas”; isso implica que a funcionalidade de sugestão deve gerar recomendações que efetivamente reflitam o histórico e as avaliações do cliente, sob risco de comprometer o objetivo de fidelização.

Os dois primeiros objetivos de prioridade alta, confiabilidade da informação de estoque e segurança dos dados e transações do cliente, não se esgotam aqui: eles voltam a aparecer no capítulo de Arquitetura, junto da discussão dos princípios SOLID, quando tratamos de como o `EstoqueService` isola a lógica de estoque (princípio da responsabilidade única) e de como os adaptadores externos isolam credenciais e dados sensíveis do restante do sistema.

Outros atributos de qualidade comuns em sistemas desse tipo, como o desempenho do totem em horário de pico, a manutenibilidade do software (também retomada no capítulo de Arquitetura) e a portabilidade entre totem físico e aplicativo móvel, não são detalhados

neste capítulo pelos mesmos motivos apontados na seção anterior, e ficam fora do escopo de detalhamento deste trabalho acadêmico.

Capítulo 2

Arquitetura do Sistema

Este capítulo apresenta a arquitetura proposta para o Sistema Unespão e as decisões que a sustentam. A ideia aqui não é repetir o que já foi dito na introdução sobre o problema, mas mostrar como o sistema foi estruturado para resolvê-lo: quais são suas fronteiras, que estilo arquitetural guia a organização do código, quais são as grandes peças que o compõem e por que certas escolhas técnicas foram tomadas em detrimento de outras. A base para esta seção é o levantamento arquitetural feito pela própria equipe ao longo do semestre, documentado no artefato “Sistema Unespão — Documentação Arquitetural (Modelo C4 e Princípios SOLID)”, que usamos como referência de projeto e do qual reaproveitamos os diagramas C4 de Contexto e de Containers.

2.1 Contexto e fronteiras do sistema

O Sistema Unespão automatiza o atendimento de uma padaria que vende lanches personalizáveis no estilo Subway/Spoletto: o cliente parte de um produto base (um tipo de pão, por exemplo) e vai adicionando os ingredientes que quiser, montando um item personalizado que compõe seu pedido. Essa interação acontece por dois pontos de acesso – um totem físico instalado na loja e o navegador do celular do cliente –, detalhados na seção de Contêineres a seguir, e o pedido só é considerado concluído quando o pagamento é processado.

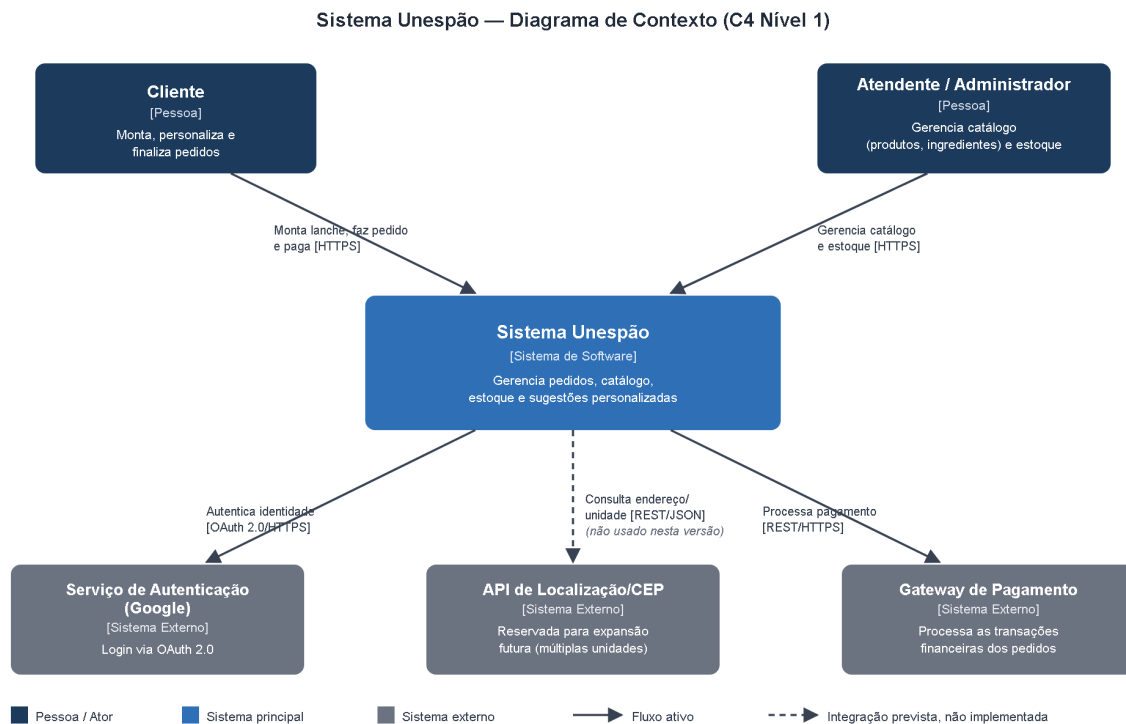


Figura 2.1: Diagrama de Contexto do Sistema Unespão (C4 Nível 1), já com os dois atores humanos do sistema.

A Figura 2.1 traz o diagrama de contexto (C4 Nível 1), que trata o sistema como uma caixa preta e mostra apenas quem interage com ele de fora. Nela aparecem dois atores humanos – o Cliente, que monta, personaliza e finaliza pedidos, e o Atendente/Administrador da padaria, responsável pela gestão de catálogo (produtos base e ingredientes) e de estoque –, o próprio Sistema Unespão, e três sistemas externos com os quais ele troca informação: o Serviço de Autenticação do Google, a API de Localização/CEP e o Gateway de Pagamento. O diagrama também indica os protocolos de cada relação – o cliente fala com o sistema via HTTPS, e o sistema, por sua vez, usa HTTPS/OAuth 2.0 para autenticar o cliente junto ao Google, REST/JSON para consultar a API de CEP e REST/HTTPS para acionar o gateway de pagamento. Em resumo, a lógica de negócio inteira mora dentro da caixa “Sistema Unespão”; os três sistemas externos são fornecedores de capacidades específicas (identidade, endereço, dinheiro) que o Unespão consome, nunca o contrário.

A versão do diagrama de contexto produzida na iteração anterior da documentação arquitetural mostrava apenas o Cliente, de antes de o grupo formalizar o Atendente/Administrador como ator do sistema. A Figura 2.1 já foi atualizada para refletir os dois atores.

O sistema modela uma única loja da padaria Unespão, não uma rede de unidades. A API de Localização/CEP aparece no diagrama pensando numa expansão futura do produto – o dia em que a Unespão eventualmente abrir mais lojas e precisar direcionar o cliente para a unidade correta –, mas hoje ela permanece fora de uso efetivo; sua menção fica registrada no diagrama para essa eventualidade.

Ficam fora das fronteiras do sistema, por decisão de escopo do grupo: a logística de entrega (o modelo é de retirada na própria loja, via totem ou app, sem fluxo de delivery); a gestão financeira e contábil da padaria como negócio (o sistema processa o pagamento pontual de cada pedido através do gateway, mas não é um sistema de fluxo de caixa ou contabilidade); a gestão de funcionários e recursos humanos (folha de pagamento, escalas de trabalho); e o atendimento presencial que eventualmente aconteça em paralelo ao canal digital, por ser operação humana da padaria e não uma funcionalidade de software.

2.2 Estilo ou padrão arquitetural

Optamos por Clean Architecture, organizada como uma arquitetura em camadas com fluxo unidirecional de dependências. Na prática, o backend é dividido em camadas concêntricas – Controllers, Services, Repositories/External Adapters e Domain Models – e cada camada só pode depender da camada imediatamente abaixo dela, nunca o contrário. As entidades de domínio (Cliente, Pedido, Estoque, e assim por diante) não conhecem nenhuma camada superior; quem depende delas são os repositórios e os serviços, não o inverso.

A escolha está ligada aos objetivos de qualidade definidos no capítulo de Introdução. A confiabilidade da informação de estoque, por exemplo, exige que a lógica de baixa de insumos fique concentrada num único lugar e não espalhada por controllers ou por chamadas ad hoc ao banco – é o que o Princípio da Responsabilidade Única (SRP) garante ao isolar essa lógica dentro de um `EstoqueService` dedicado, único ponto do sistema autorizado a decidir como o estoque é debitado após um pedido. Já o objetivo de segurança dos dados e das transações do cliente se apoia no Princípio da Inversão de Dependência (DIP): os serviços de negócio, como o `PedidoService`, dependem apenas de interfaces (`IGatewayPagamentoAdapter`, `IPedidoRepository`) e nunca de implementações concretas, o que mantém credenciais de integrações externas – tokens do Google, chaves do gateway de pagamento – isoladas dentro dos adapters, sem vazamento para as camadas de negócio.

O capítulo de Projeto de Componentes retoma cada princípio SOLID com mais profundidade, classe por classe; aqui a escolha aparece como decisão de alto nível porque orienta a arquitetura descrita nas seções seguintes.

2.3 Contêineres e componentes

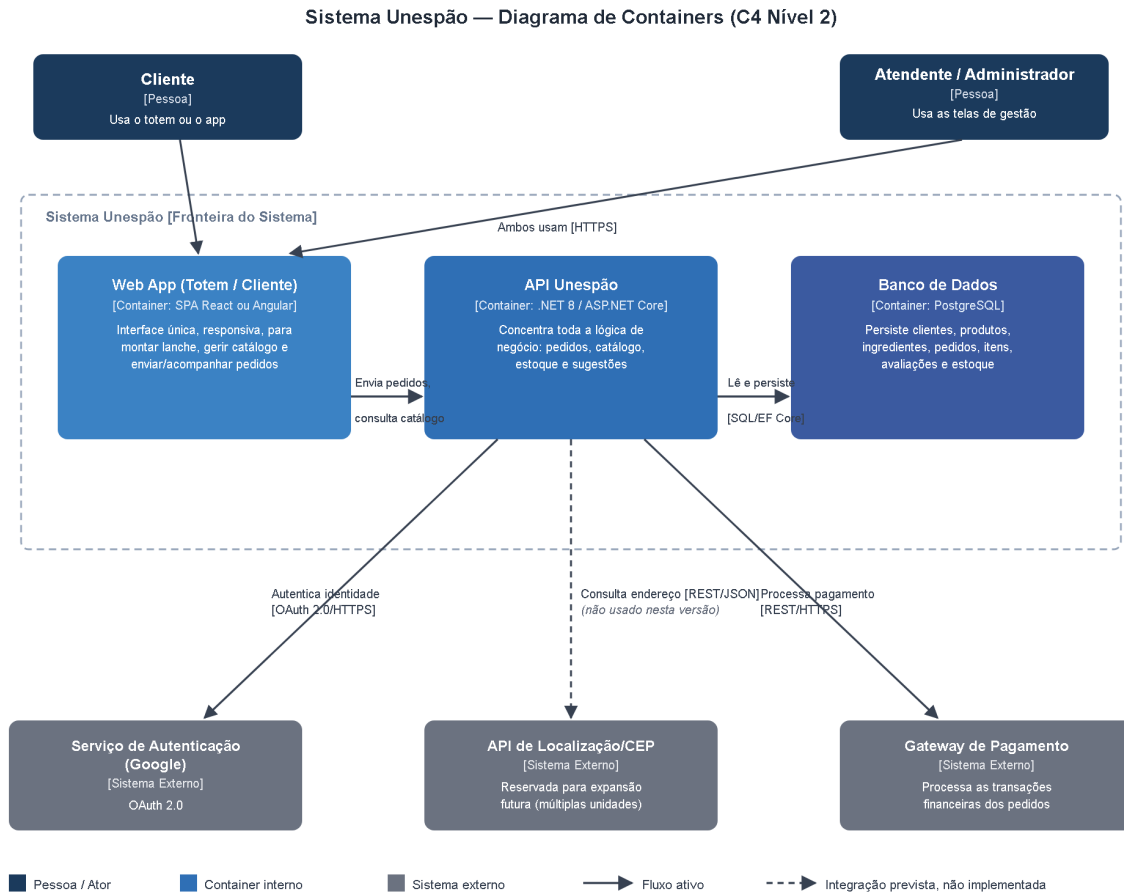


Figura 2.2: Diagrama de Containers do Sistema Unespão (C4 Nível 2).

A Figura 2.2 mostra o diagrama de containers (C4 Nível 2), que abre a caixa preta do diagrama anterior e revela as três peças tecnológicas que formam o Sistema Unespão.

O primeiro container é o Web App (Totem e Cliente): um único SPA construído em React ou Angular (a escolha definitiva entre as duas stacks fica para a equipe fechar durante o desenvolvimento), rodando em modo responsivo tanto na tela do totem físico quanto no navegador do cliente, sem app móvel separado nem código duplicado entre os dois pontos de acesso. É essa mesma interface, adaptada ao tamanho de tela de cada dispositivo, que tanto o Cliente quanto o Atendente/Administrador usam, com permissões diferenciadas por perfil controladas pela API: o Cliente monta o lanche, consulta o cardápio disponível (já refletindo o estoque) e finaliza o pedido, enquanto o Atendente/Administrador acessa as telas de gestão de catálogo e de estoque. O segundo container é a API Unespão, escrita em .NET 8 com ASP.NET Core, que concentra toda a lógica de negócio do sistema – cadastro de produtos e ingredientes, composição de itens personalizados, controle de estoque, cancelamento ou edição de item do pedido antes da confirmação do pagamento, histórico de pedidos e geração de sugestões personalizadas. O terceiro é o banco de dados PostgreSQL, responsável por persistir clientes, produtos base, ingredientes, pedidos, itens personalizados, avaliações e estoque.

A comunicação entre esses containers segue um padrão importante para a segurança do sistema: o Web App fala com a API Unespão via HTTPS/JSON, e é só a API que acessa o banco (via Entity Framework Core com o provider Npgsql) e que se comunica com os três sistemas externos do diagrama de contexto. Nem o Google Auth, nem a API de CEP, nem o

gateway de pagamento são acessados diretamente pelo frontend – toda integração externa passa pela API, que guarda tokens e credenciais fora do alcance do navegador do cliente, viabilizando o objetivo de segurança mencionado na seção anterior.

Do ponto de vista de componentes internos da API (C4 Nível 3, detalhado no capítulo de Projeto de Componentes), a organização segue seis grupos alinhados ao estilo em camadas: Controllers na porta de entrada HTTP, Services concentrando as regras de negócio, Repositories abstraindo a persistência, External Adapters isolando a comunicação com sistemas de terceiros, Domain Models representando as entidades ricas do domínio, e a camada de Infrastructure (o `UnespaoDbContext`) mapeando esses modelos para o esquema relacional do PostgreSQL.

2.4 Decisões arquiteturais relevantes

Tabela 2.1: Decisões arquiteturais relevantes do Sistema Unespão.

Decisão	Motivação	Consequências
Adotar Clean Architecture / arquitetura em camadas com fluxo unidirecional de dependências	Isolar a lógica de negócio de detalhes de infraestrutura, facilitando testes e reduzindo o custo de trocar uma peça tecnológica sem afetar as demais.	Mais interfaces e classes de mapeamento entre camadas; em compensação, cada camada é testável isoladamente e trocar banco ou gateway fica restrito a Repositories e External Adapters.
Centralizar toda comunicação com sistemas externos na API Unespão, nunca no frontend	Evitar que tokens de autenticação e credenciais de pagamento fiquem expostos no navegador/totem do cliente.	Todo fluxo de autenticação e pagamento passa por uma chamada adicional ao backend, o que acrescenta uma etapa de rede mas concentra a superfície de segurança em um único ponto auditável.
Usar um único SPA responsivo (React ou Angular) em vez de app móvel nativo separado (ver seção de Contêineres)	Atender totem e navegador do cliente com o mesmo frontend e backend, sem duplicar código.	Toda regra de negócio fica no backend, evitando inconsistência entre o que aparece no totem e no navegador.
Modelar uma loja única, mantendo a API de CEP apenas como integração prevista (ver seção de Contexto)	O escopo do projeto cobre uma padaria só.	A integração de CEP não é implementada nesta versão; sua menção evita redesenhar o contexto numa expansão futura.
Isolar a lógica de estoque em um EstoqueService dedicado (SRP)	Garantir a confiabilidade da formação de estoque, objetivo de qualidade de prioridade alta definido na Introdução.	Único ponto de responsabilidade para a baixa de insumos, reduzindo o risco de inconsistência entre o cardápio exibido e o estoque físico.
Depender de abstrações nos serviços de negócio, isolando credenciais nos External Adapters (DIP)	Suportar a segurança dos dados e transações do cliente sem acoplar a lógica de pedido a uma implementação específica de gateway ou autenticação.	Facilita testes com dependências fake e permite trocar/adicionar um gateway de pagamento criando um novo adapter, sem alterar o PedidoService.

2.5 Restrições de arquitetura

A stack tecnológica já está definida e não é objeto de discussão neste projeto: backend em .NET 8 com ASP.NET Core, persistência em PostgreSQL e o frontend SPA descrito na seção de Contêineres. Trata-se de uma restrição herdada da documentação arquitetural produzida anteriormente pelo próprio grupo e mantida como decisão de projeto consolidada.

O sistema é desenvolvido no contexto da disciplina de Engenharia de Software II, na UNESP, o que traz restrições próprias de um trabalho acadêmico: prazo de entrega fixo dentro do semestre letivo, equipe pequena e sem dedicação exclusiva ao projeto, e a exigência de que a documentação evidencie explicitamente os conceitos vistos em aula (Modelo C4, princípios SOLID, entre outros), não apenas o funcionamento do software em si.

Há também três restrições de integração obrigatórias, não triviais de contornar sem redesenhar a arquitetura: autenticação via Google OAuth 2.0, processamento financeiro via gateway de pagamento externo, e a API de CEP reservada para a expansão a múltiplas lojas já discutida nas seções anteriores. As três partem exclusivamente da API Unespão, nunca do frontend, pela mesma decisão de segurança da tabela acima.

Capítulo 3

Projeto de Componentes

3.1 Visão geral dos componentes

A API Unespão (.NET 8 / ASP.NET Core) é o único container que concentra lógica de negócio no sistema: o Web App/Totem funciona apenas como camada de apresentação, e o banco PostgreSQL só é acessado através dela. Internamente essa API está organizada em seis grupos de componentes, com fluxo de dependência unidirecional – Controllers chamam Services, que por sua vez dependem de Repositories e de External Adapters; ambos operam sobre os Domain Models, e a persistência efetiva passa pela camada de Infrastructure (DbContext). Essa organização segue os princípios de Clean Architecture: cada camada conhece apenas a camada imediatamente abaixo dela, e os Domain Models não dependem de nenhuma outra camada.

A tabela a seguir resume os componentes e suas responsabilidades.

Tabela 3.1: Principais componentes da API Unespão.

Componente	Responsabilidade principal
Controllers (ClientesController, PedidosController, ProdutosController, AvaliacoesController, EstoqueController)	Receber requisições HTTP do frontend, validar dados de entrada, delegar a execução aos serviços apropriados e devolver respostas em JSON. Não contém regra de negócio.
Services (ClienteService, ProdutoService, Pedi- doService, Avaliacao- Service, EstoqueService, SugestaoPersonalizada- Service)	Concentrar a lógica de domínio, coordenar chamadas a repositórios e adapters externos e garantir a consistência de operações que envolvem mais de uma entidade (por exemplo, criar um pedido e dar baixa no estoque).
Repositories (IRepository<T> e as interfaces específicas IClienteRepository, IPedidoRepository, IProdutoRepository, IAvaliacaoRepository, IEstoqueRepository, com suas implementações concretas)	Abstrair o acesso a dados, expondo uma interface padronizada de operações CRUD sobre as entidades de domínio, usando Entity Framework Core e o provider Npgsql.
External Adapters (GoogleAuthAdapter, LocalizacaoAdapter, Ga- tewayPagamentoAdapter)	Isolar a comunicação com sistemas externos (autenticação Google, consulta de CEP e gateway de pagamento), traduzindo o formato exigido por cada API externa para o modelo interno da aplicação.
Domain Models (Cliente, ProdutoBase, Ingrediente, ItemPersonalizado, Pedido, AvaliacaoPrato, Estoque)	Representar as entidades de negócio e suas regras, sem depender de nenhuma outra camada da aplicação.
Infrastructure (UnespaoDbContext)	Mapear os Domain Models para o esquema relacional do PostgreSQL e gerenciar a conexão com o banco via Entity Framework Core.

Do ponto de vista de quem usa o sistema, os dois atores do minimundo acionam grupos distintos de Controllers: o Cliente interage com `ClientesController`, `PedidosController` e `AvaliacoesController` (cadastro, autenticação, montagem/edição de pedido personalizado, pagamento e avaliação de pratos); já o Atendente/Administrador da padaria – ator de perfil mais simples, responsável pelo CRUD de catálogo e estoque – opera por meio de `ProdutosController` e `EstoqueController`. Essa separação de Controllers por ator já reflete, na camada de apresentação da API, a distinção de perfis que motiva a existência do `EstoqueService` como ponto único de manipulação de estoque, detalhado na seção seguinte.

Optamos por manter essa granularidade, com camadas bem definidas em vez de uma organização mais “achatada”, porque ela conversa diretamente com dois dos objetivos de qualidade definidos no capítulo de Introdução. Um é a confiabilidade da informação de estoque, que depende de a lógica de baixa de insumos estar concentrada e isolada em um único componente

(`EstoqueService`). O outro é a segurança dos dados e transações do cliente, que exige que credenciais de terceiros (Google, gateway de pagamento) fiquem restritas aos Adapters e nunca cheguem ao frontend.

3.2 Detalhamento dos componentes principais

Dos seis grupos, três concentram a maior parte da complexidade e do risco de mudança do sistema: os Services, os Repositories e os External Adapters. É neles que detalhamos a seguir responsabilidades, interfaces e dependências. Como o Sistema Unespão é um projeto orientado a objetos, o diagrama de classes UML simplificado abaixo resume os grupos de componentes e as principais interfaces e dependências entre eles; o texto que segue detalha cada um.

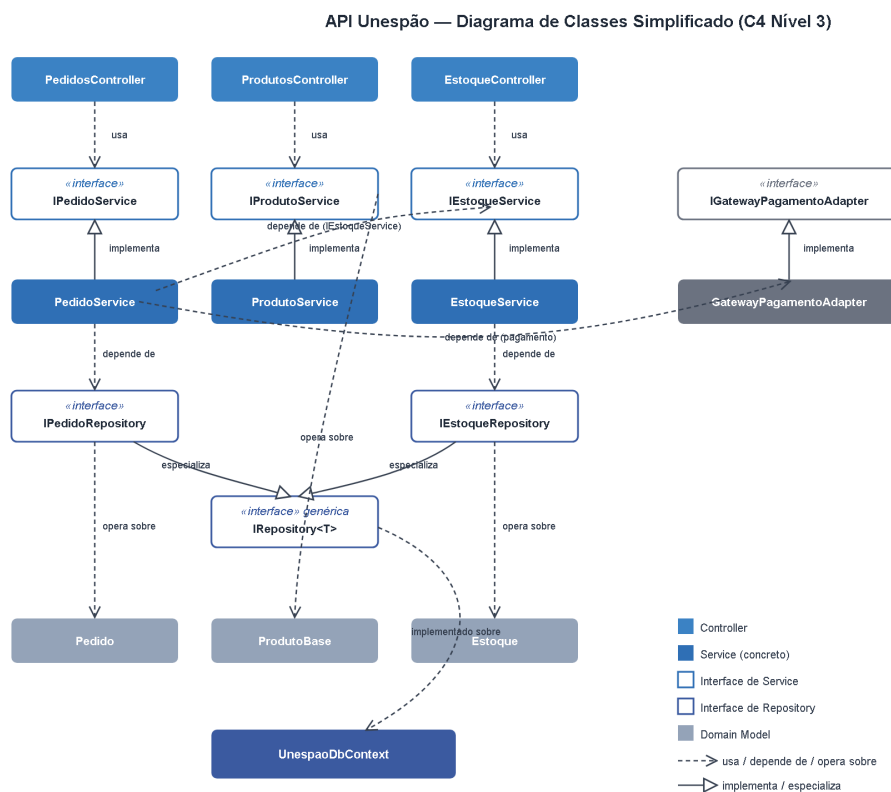


Figura 3.1: Diagrama de classes simplificado da API Unespão (C4 Nível 3), com os grupos de componentes mais relevantes e suas dependências.

Na leitura do diagrama, as setas tracejadas com ponta aberta (“usa”/“depende de”) indicam dependência em tempo de execução via injeção de construtor, enquanto as setas de implementação de interface ligam cada Service à sua abstração correspondente – é essa cadeia (Controller → interface de Service → Service concreto → interface de Repository/Adapter) que materializa a Clean Architecture descrita na seção anterior: nenhuma seta cruza diretamente para uma classe concreta de outra camada.

PedidoService. É o componente mais crítico do sistema, pois orquestra o ciclo de vida completo de um pedido: criação, edição ou cancelamento de itens antes da confirmação de pagamento (requisito adotado pelo grupo), personalização dos itens e processamento do pagamento. Suas dependências são todas abstrações – `IPedidoRepository`, `IEstoqueService` e `IGatewayPagamentoAdapter` – nunca as classes concretas correspondentes. Essa decisão não

é incidental: é o que permite trocar o gateway de pagamento ou substituir o repositório por uma versão em memória nos testes sem tocar em uma linha do serviço.

Para tornar concreto esse fluxo de criação/confirmação de pedido – o mais crítico do sistema –, o diagrama de sequência a seguir mostra a interação entre `PedidosController`, `PedidoService` e as abstrações que ele consome, sem detalhar o Domain Model interno:

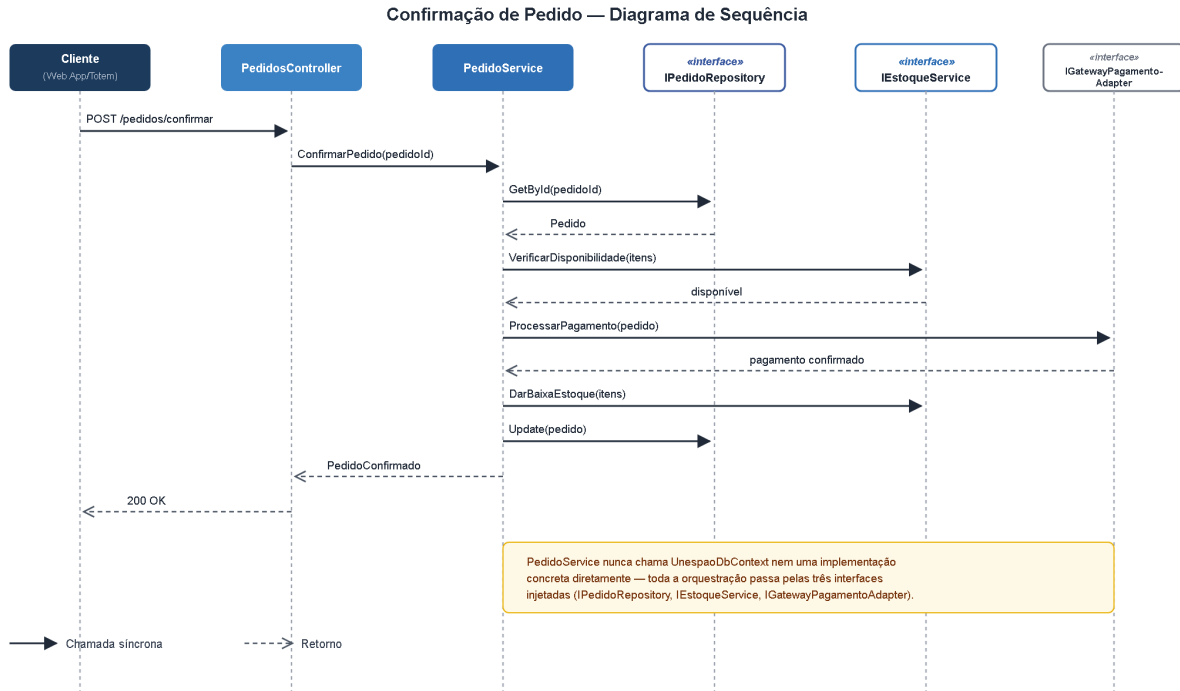


Figura 3.2: Diagrama de sequência da confirmação de pedido, do `PedidosController` até as três interfaces que o `PedidoService` orquestra.

O diagrama evidencia que `PedidoService` nunca chama diretamente o `UnespaDbContext` nem uma implementação concreta de gateway: toda a orquestração passa pelas três interfaces injetadas, o que é o que possibilita, por exemplo, substituir `GatewayPagamentoAdapter` por um dublê de teste sem alterar essa sequência de chamadas.

EstoqueService. Cuida exclusivamente da disponibilidade de produtos base e ingredientes, incluindo a baixa de insumos quando um pedido é confirmado. A separação desse componente do `PedidoService` é uma decisão deliberada de projeto: como o objetivo de qualidade “confiabilidade da informação de estoque” tem prioridade alta, isolar essa lógica em uma única classe reduz a chance de uma alteração na regra de pedidos afetar, por efeito colateral, o controle de estoque, e vice-versa. Na leitura do grupo, esse componente também é o ponto natural de extensão para os requisitos de CRUD de catálogo e estoque operados pelo Atendente/Administrador: as operações de cadastro e ajuste de estoque desse ator, expostas via `ProdutosController` e `EstoqueController`, passam pelo mesmo `EstoqueService`, e não por um caminho paralelo.

IRepository<T> e as implementações concretas. A camada de Repositories é construída sobre uma interface genérica, `IRepository<T>`, que define as operações CRUD comuns a todas as entidades, complementada por interfaces específicas (`IClienteRepository`, `IPedidoRepository` etc.) que acrescentam métodos próprios do domínio, como `GetClienteWithHistorico`. Cada Service depende apenas da interface, nunca da implementação concreta que fala com o PostgreSQL, o que possibilita a existência de implementações alternativas para teste, como `MockPedidoRepository`, discutida no capítulo de Testes. Essa relação de especialização (`IPedidoRepository` e `IEstoqueRepository` estendendo `IRepository<T>`) é

exatamente o que o diagrama de classes acima representa pelas setas de generalização entre essas interfaces.

External Adapters. `GoogleAuthAdapter`, `LocalizacaoAdapter` e `GatewayPagamentoAdapter` encapsulam, cada um, a comunicação com um sistema externo específico do C4 Nível 1 (Serviço de Autenticação Google, API de Localização/CEP e Gateway de Pagamento, respectivamente). Nenhum deles é chamado diretamente pelo frontend: a API Unespão sempre medeia essas integrações, o que mantém tokens e credenciais fora do navegador do cliente. Vale registrar que, como decidido no escopo do produto, a API de Localização/CEP é um recurso pensado para uma futura expansão a múltiplas unidades; no escopo atual, de loja única, o `LocalizacaoAdapter` existe na arquitetura mas não tem uso funcional imediato.

UnespaoDbContext. Fecha o fluxo de dependências mapeando os Domain Models para o esquema relacional via Entity Framework Core, expondo um `DbSet<>` para cada entidade persistente (Clientes, ProdutosBase, Ingredientes, Pedidos, ItensPersonalizados, AvaliacoesPrato e Estoque). É o único ponto do sistema que conhece o dialeto do PostgreSQL; substituir o banco de dados por outro SGBD afetaria, em tese, apenas essa classe e as implementações concretas dos Repositories.

3.3 Princípios de projeto

Os cinco princípios SOLID orientaram as decisões de projeto da API Unespão. Em vez de tratá-los como uma lista teórica, procuramos, para cada um, indicar o problema concreto de projeto que ele resolveu no sistema, complementando a análise com o trecho de código do construtor de `PedidoService` a seguir, que evidencia de forma direta a aplicação de DIP e ISP:

```
1 public class PedidoService : IPedidoService
2 {
3     private readonly IPedidoRepository _pedidoRepository;
4     private readonly IEstoqueService _estoqueService;
5     private readonly IGatewayPagamentoAdapter _gatewayPagamento;
6
7     public PedidoService(
8         IPedidoRepository pedidoRepository,
9         IEstoqueService estoqueService,
10        IGatewayPagamentoAdapter gatewayPagamento)
11    {
12        _pedidoRepository = pedidoRepository;
13        _estoqueService = estoqueService;
14        _gatewayPagamento = gatewayPagamento;
15    }
16
17    // ConfirmarPedido, AdicionarItem, RemoverItem, etc.
18 }
```

Listing 3.1: Construtor de `PedidoService`, com suas três dependências expressas como interfaces.

Nenhum dos três parâmetros do construtor é uma classe concreta (`PedidoRepository`, `EstoqueService` ou `GatewayPagamentoAdapter`): todos são interfaces, resolvidas em tempo de execução pelo container de injeção de dependência do ASP.NET Core. É esse trecho – junto do diagrama de classes da seção anterior – que sustenta concretamente as discussões de DIP e ISP abaixo.

Responsabilidade única (SRP). Cada Service tem um único motivo para mudar. `PedidoService` cuida só do ciclo de vida do pedido; `EstoqueService`, só de estoque e baixa de insumos; `AvaliacaoService`, só de feedback de pratos; `SugestaoPersonalizadaService`, só de recomendações. O ganho prático é direto: se a regra de pontuação de uma avaliação mudar, apenas `AvaliacaoService` é tocado, sem risco de regressão em pedidos ou estoque. Os Controllers seguem a mesma lógica, cada um respondendo a um conjunto coeso de endpoints.

Aberto/fechado (OCP). A interface genérica `IRepository<T>` permite acomodar novas entidades sem alterar código já existente, bastando criar uma nova implementação. O caso

mais ilustrativo, porém, é o dos adapters de pagamento: trocar o gateway de Stripe por PagSeguro, ou adicionar suporte a Pix, significa criar uma nova classe que implementa `IGatewayPagamentoAdapter` – o mesmo parâmetro `gatewayPagamento` do construtor acima passaria a receber outra implementação –, sem que `PedidoService` precise mudar uma única linha. Essa substituíbilidade de gateways é, na prática, a aplicação mais concreta de OCP no sistema, e se aproxima do espírito do padrão Strategy – retomamos esse ponto na seção de padrões de projeto.

Substituição de Liskov (LSP). Qualquer implementação de `IPedidoRepository` precisa poder substituir outra sem que `PedidoService` perceba diferença de comportamento. É o que garante, por exemplo, que `PedidoRepository` (a implementação real sobre PostgreSQL) e `MockPedidoRepository` (uma implementação em memória usada em teste) sejam intercambiáveis do ponto de vista do serviço que os consome – ambas encaixam no mesmo parâmetro `pedidoRepository` do construtor. O cuidado aqui é que as implementações não podem quebrar as garantias do contrato: se `IPedidoRepository.GetById` promete devolver um `Pedido` válido, toda implementação precisa honrar isso, sob risco de falhas silenciosas em produção que só apareceriam ao trocar a implementação usada.

Segregação de interfaces (ISP). Em vez de uma interface única e genérica concentrando toda a lógica de serviço, o sistema define interfaces dedicadas – `IClienteService`, `IPedidoService`, `IAvaliacaoService`, `IEstoqueService`, `ISugestaoPersonalizadaService` – cada uma expondo apenas os métodos relevantes ao seu domínio. O mesmo raciocínio se aplica aos repositórios: `IRepository<T>` cobre o CRUD genérico, e interfaces como `IClienteRepository` acrescentam apenas os métodos específicos daquela entidade. O efeito é que cada Controller depende só do que usa (`ClientesController` não é forçado a conhecer métodos de estoque ou de pagamento), o que mantém o acoplamento baixo e permite que cada interface evolua de forma independente. No construtor de `PedidoService` isso aparece como três parâmetros estreitos e específicos (`IPedidoRepository`, `IEstoqueService`, `IGatewayPagamentoAdapter`), em vez de uma única interface genérica de “serviços” que agregasse métodos não usados por esse Service.

Inversão de dependência (DIP). `PedidoService` não conhece nenhuma classe concreta: suas dependências são sempre interfaces – `IPedidoRepository`, `IEstoqueService`, `IGatewayPagamentoAdapter` –, exatamente como declarado no construtor acima, nunca `PedidoRepository`, `EstoqueService` ou `GatewayPagamentoAdapter` diretamente. Quem resolve essas dependências em tempo de execução é o container de injeção de dependência do ASP.NET Core, que registra as implementações concretas como provedoras de cada interface e as injeta no construtor de cada Service. Esse é, possivelmente, o princípio com maior impacto estrutural no projeto: é ele que sustenta a testabilidade (basta injetar uma implementação fake), a possibilidade de trocar o banco de dados afetando só a camada de Repositories, e o baixo acoplamento entre todas as camadas descritas na seção anterior.

3.4 Padrões de projeto

Documentamos aqui apenas os padrões cuja aplicação está de fato evidenciada na arquitetura do sistema.

Repository. Problema: a lógica de negócio dos Services não deveria depender diretamente de detalhes de acesso a dados (Entity Framework Core, SQL, provider Npgsql). Solução adotada: uma interface genérica `IRepository<T>` com as operações CRUD comuns, especializada por interfaces como `IPedidoRepository` e `IClienteRepository` para métodos próprios de cada entidade, cada uma com sua implementação concreta sobre PostgreSQL. Participantes: os Services (clientes do padrão), as interfaces de repositório e suas implementações concretas, e o `UnespaDbContext` por trás delas. Benefício obtido: a lógica de negócio permanece isolada da

tecnologia de persistência, e a substituição de uma implementação real por uma implementação de teste em memória (`MockPedidoRepository`) se torna possível sem alterar o código que a consome, o que viabiliza a estratégia de testes de unidade descrita no capítulo correspondente.

Adapter. Problema: o sistema precisa se comunicar com três serviços externos heterogêneos (autenticação Google via OAuth 2.0, API de Localização/CEP via REST/JSON, Gateway de Pagamento via REST/HTTPS), cada um com seu próprio protocolo e formato de dados, sem que essa heterogeneidade vaze para a lógica de negócio. Solução adotada: três classes – `GoogleAuthAdapter`, `LocalizacaoAdapter` e `GatewayPagamentoAdapter` – que traduzem as chamadas da aplicação para o formato exigido por cada API externa e traduzem de volta as respostas para o modelo interno. Participantes: os Services que consomem os adapters (por exemplo, `PedidoService` consumindo `IGatewayPagamentoAdapter`) e as classes adapter propriamente ditas. Benefício obtido: o sistema fica isolado de mudanças nesses serviços externos; se o gateway de pagamento mudar sua API, o impacto fica contido no adapter correspondente, sem se propagar para `PedidoService` ou para os Controllers.

Padrão Adapter aplicado ao processamento de pagamento

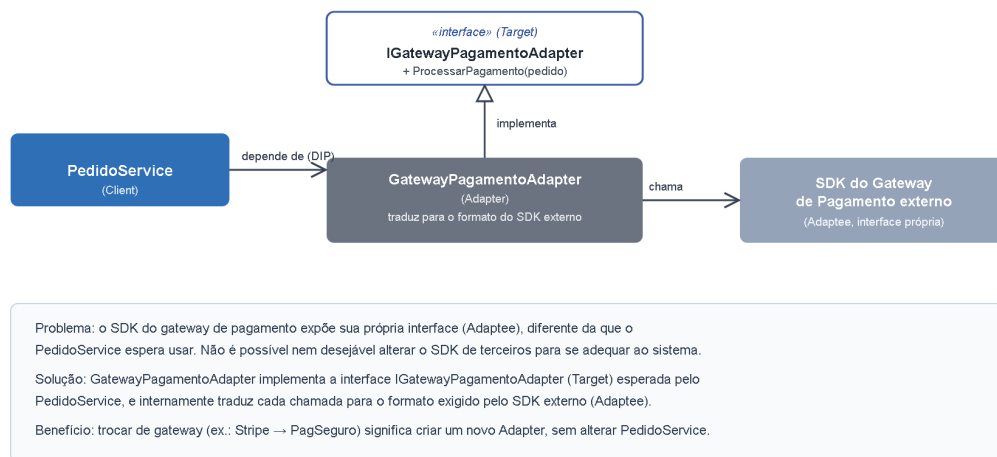


Figura 3.3: Estrutura clássica do padrão Adapter (Target/Adapter/Adaptee) instanciada com as classes reais do caso de pagamento: `PedidoService` (Client) depende apenas de `IGatewayPagamentoAdapter` (Target); `GatewayPagamentoAdapter` implementa essa interface e traduz cada chamada para o SDK do gateway externo (Adaptee), cuja interface própria o sistema não controla.

Injeção de Dependência (via container do ASP.NET Core). Problema: se cada Service instanciasse diretamente suas dependências concretas (o repositório do EF Core, o adapter HTTP do gateway de pagamento), o acoplamento entre camadas inviabilizaria tanto a testabilidade quanto a substituição de implementações discutidas no DIP. Solução adotada: registrar cada interface (`IPedidoRepository`, `IEstoqueService`, `IGatewayPagamentoAdapter` etc.) e sua implementação concreta correspondente no container de injeção de dependência nativo do ASP.NET Core, deixando que o framework resolva e injete as instâncias apropriadas nos construtores dos Services e Controllers em tempo de execução – o construtor de `PedidoService` mostrado na seção anterior é exatamente o ponto de extensão que esse container preenche. Participantes: praticamente todos os componentes das camadas de Services, Repositories e External Adapters, mediados pelo container. Benefício obtido: as dependências entre camadas passam a ser configuradas externamente, e não codificadas dentro das classes, o que é a base

técnica que sustenta tanto a testabilidade (substituir uma implementação real por um mock nos testes) quanto a substituição de tecnologia (trocar de gateway de pagamento ou de banco de dados) sem alterar código consumidor.

Substituibilidade de adapters de pagamento como aplicação prática de OCP (aproximação de Strategy). Vale destacar separadamente esse caso porque, embora não seja um Strategy formalmente nomeado nas fontes do grupo, o comportamento observado se aproxima bastante desse padrão: diferentes gateways de pagamento (Stripe, PagSeguro, Pix) podem coexistir como implementações alternativas de `IGatewayPagamentoAdapter`, e o `PedidoService` opera sobre a abstração sem saber qual implementação concreta está em uso a cada momento, o que na prática equivale a poder trocar o “algoritmo” de processamento de pagamento em tempo de configuração. Registramos essa observação como uma leitura de projeto, e não como um padrão Strategy formalmente documentado nas fontes do grupo, para não sugerir uma decisão de projeto mais elaborada do que a que foi de fato tomada.

Não documentamos o padrão Decorator neste capítulo, pois não há evidência, nas fontes analisadas, de que ele seja de fato utilizado na arquitetura do Sistema Unespão. O exercício sobre Decorator presente no repositório do grupo é um modelo de como documentar um padrão de projeto produzido em outra atividade da disciplina, não parte da implementação do sistema em si.

Capítulo 4

Projeto de Interface do Usuário

Este capítulo descreve os fluxos de interação com o Sistema Unespão para os dois perfis de usuário previstos no Minimundo: o Cliente, que monta e finaliza pedidos pelo totem físico da loja ou pelo aplicativo móvel, e o Atendente/Administrador da padaria, responsável pela gestão de catálogo e estoque. Diferente dos capítulos de arquitetura, aqui o foco não é a estrutura interna do software, mas a experiência de quem está usando o sistema em cada um desses papéis.

4.1 Estado atual dos artefatos de interface

Até o momento desta versão do documento, o grupo não produziu protótipos visuais do sistema, nem em Figma nem em qualquer outra ferramenta de prototipação. O trabalho se concentrou primeiro na modelagem do domínio e na arquitetura, já documentadas nos capítulos anteriores, e a etapa de design de telas ainda não começou. Optamos, assim, por descrever os fluxos de forma textual e estrutural, sem propor layouts, paletas de cor ou composições de tela específicas – qualquer detalhe desse tipo seria invenção nesta fase do projeto e não corresponderia a uma decisão real da equipe. Quando os protótipos forem produzidos numa iteração futura, este capítulo deve ser atualizado para referenciá-los, incluindo os diagramas de atividade correspondentes aos fluxos descritos a seguir. Essa ausência de protótipos vale para os dois perfis de usuário tratados neste capítulo, Cliente e Atendente/Administrador.

Isso não quer dizer que o projeto de interface esteja em branco. As decisões estruturais do fluxo, como a ordem das etapas, o que cada uma exige do usuário e, em particular, como funciona a autenticação num dispositivo compartilhado, já foram discutidas e fixadas pelo PO, e são justamente o conteúdo deste capítulo.

4.2 Canais de interação do Cliente

O cliente acessa o sistema por um destes dois canais, que compartilham a mesma lógica de negócio no backend (API Unespão) e diferem principalmente na forma de entrada:

- **Totem físico**, instalado na loja, pensado para o cliente que já está presencialmente na padaria e decide o pedido ali mesmo. É um dispositivo compartilhado entre vários clientes ao longo do dia.
- **Aplicativo móvel**, no celular do próprio cliente, usado tanto para pedir remotamente quanto para complementar a experiência do totem – por exemplo, quando o cliente já chega com uma conta autenticada.

Os passos do pedido (escolha do produto, personalização, pagamento) são conceitualmente os mesmos nos dois canais. A diferença que importa do ponto de vista de projeto de interface está na etapa de autenticação, tratada separadamente mais adiante.

4.3 Fluxo principal de interação do cliente

O fluxo de ponta a ponta, do início do pedido até a etapa posterior de avaliação, segue as seguintes etapas:

1. **Identificação no canal.** No aplicativo, o cliente autentica normalmente – contas de clientes recorrentes têm histórico de pedidos associado, conforme o Minimundo. No totem, a identificação segue a decisão de projeto detalhada na próxima seção: QR code vinculando a sessão do app, ou pedido anônimo com vínculo posterior.
2. **Escolha do produto base.** O cliente seleciona o item que servirá de ponto de partida para a personalização, por exemplo um tipo de pão ou uma base de salada, dentre os produtos disponíveis em estoque.
3. **Adição de ingredientes com atualização incremental do preço.** A cada ingrediente adicionado ou removido, o valor do item é recalculado e exibido imediatamente, sem que o cliente precise avançar de tela para saber o custo da personalização. Essa atualização incremental existe porque o preço final de um Item Personalizado só é conhecido depois da composição completa de Ingredientes, e faz sentido que o cliente acompanhe esse custo enquanto ainda está decidindo: evita surpresa na etapa de revisão e reduz a chance de abandono do pedido.
4. **Revisão do pedido.** Antes de seguir para o pagamento, o cliente vê o resumo dos itens personalizados escolhidos, com a possibilidade de cancelar ou editar qualquer item. Este ponto está alinhado ao requisito funcional já definido pelo PO de permitir alteração do pedido antes da confirmação do pagamento.
5. **Pagamento.** O pedido é encaminhado ao Gateway de Pagamento via API Unespão, nunca diretamente do frontend, como descrito na arquitetura, e o cliente acompanha o status da transação.
6. **Confirmação.** Com o pagamento aprovado, o pedido é registrado e o cliente recebe a confirmação, encerrando o ciclo ativo de compra.
7. **Avaliação do prato (momento posterior).** Depois de retirar o pedido, o cliente pode registrar uma Avaliação de Prato: nota e comentário sobre o Item Personalizado consumido.
8. **Sugestões personalizadas em pedidos futuros.** Em pedidos seguintes, o sistema usa o Histórico de Pedidos e as avaliações anteriores para propor combinações que o cliente tende a gostar, reduzindo o número de decisões que ele precisa tomar do zero a cada visita.

Essas oito etapas, incluindo a bifurcação de identificação no totem (QR code vs. pedido anônimo) detalhada na seção seguinte, ainda não foram representadas em um diagrama de atividade – essa figura fica para a iteração futura em que os protótipos de interface também forem produzidos (ver seção anterior). O fluxo é o mesmo nos dois canais a partir do passo 2; a diferença está inteiramente na etapa 1, onde o totem bifurca entre a rota de QR code e a rota anônima. Quando essa figura for produzida, a rota de “pedido anônimo” deve indicar que o vínculo com a conta, quando feito, pode ocorrer depois da compra, permitindo que o histórico e as avaliações do pedido anônimo passem a contar retroativamente para o cliente.

4.4 Decisão de projeto: autenticação no totem físico

Um ponto que exigiu decisão explícita do PO foi como identificar o cliente no totem, já que esse dispositivo é compartilhado por múltiplos clientes ao longo do dia, diferente do celular no app, que é de uso pessoal.

Duas alternativas foram descartadas antes de chegar à decisão final. A primeira era exigir um fluxo completo de OAuth 2.0, o mesmo usado pelo Serviço de Autenticação do Google já presente na arquitetura, diretamente no totem. Isso obrigaria o cliente a digitar credenciais num teclado público, o que é ruim tanto em usabilidade (login lento, digitação incômoda numa tela touch) quanto em segurança, pela exposição da senha ou do token de sessão num equipamento compartilhado: se a sessão de um cliente não fosse encerrada corretamente, o próximo cliente poderia herdar o carrinho ou o histórico de quem usou o totem antes dele. A segunda alternativa cogitada foi autenticar apenas por CPF, descartada por não constituir autenticação forte (qualquer pessoa que soubesse o CPF de outra poderia se passar por ela) e por não ter respaldo em nenhuma das fontes do projeto.

A solução adotada combina duas rotas, e o cliente escolhe entre elas no início do fluxo do totem:

- **QR code vinculado à sessão do app.** O cliente abre o aplicativo no próprio celular, já autenticado ali, e usa essa sessão para gerar um QR code que o totem lê. A partir daí, o totem apenas associa o pedido em andamento à conta já logada no celular: nenhuma credencial é digitada no dispositivo público, e a sessão fica atrelada ao aparelho do cliente, não ao totem.
- **Pedido anônimo, com vínculo posterior.** Se o cliente preferir não usar o app naquele momento, o totem permite montar e pagar o pedido sem identificação alguma. Depois, ele pode vincular esse pedido à sua conta, por exemplo abrindo o app e associando o comprovante ou pedido recente, passando a contar para o Histórico de Pedidos e para as Sugestões Personalizadas.

Essa decisão evita reintroduzir no totem o mesmo risco que motivou o descarte do OAuth completo, a sessão cruzada entre clientes num equipamento compartilhado, e ainda preserva o benefício de histórico e recomendação para quem quiser se identificar. O custo é aceitar que uma parcela dos pedidos feitos no totem fique anônima, sem entrar no cálculo de Sugestões Personalizadas, a menos que o cliente faça questão de vincular depois.

Um diagrama isolando essa bifurcação – detalhando o que cada rota exige do dispositivo e da conta – também fica para a iteração futura em que os protótipos de interface forem produzidos, junto do diagrama de atividade mencionado na seção anterior.

4.5 Interface do Atendente/Administrador

Além do Cliente, o sistema prevê um segundo perfil de usuário: o Atendente/Administrador da padaria, responsável pela gestão de catálogo (produtos base e ingredientes) e de estoque. Esse é um perfil de uso interno, operado por quem trabalha na padaria, e não pelo cliente final – por isso seu canal de acesso é tratado separadamente do totem e do aplicativo do cliente.

Diferente do Cliente, não há indicação nas fontes do projeto (Minimundo ou arquitetura) de um dispositivo dedicado para esse perfil. O caminho mais plausível, dado que a arquitetura já prevê um frontend web (SPA React ou Angular) consumindo a mesma API Unespão, é que o Atendente/Administrador acesse um painel administrativo web, separado da experiência de totem/app do cliente, autenticado com suas próprias credenciais de funcionário. Como

já indicado, também aqui não há telas prototipadas; o que segue é uma descrição textual e estrutural das operações que ele precisa suportar, sem propor layout algum.

A interface desse perfil é estruturalmente mais simples que a do Cliente: não envolve personalização de pedido, atualização incremental de preço, pagamento ou avaliação de prato. Suas operações centrais são de CRUD (criação, leitura, atualização e remoção) sobre duas famílias de dados:

- **Catálogo.** Cadastro e manutenção dos Produtos Base (por exemplo, tipos de pão ou bases de salada) e dos Ingredientes disponíveis para personalização, incluindo os dados que o Cliente vê no momento de montar seu pedido, como nome, descrição e preço unitário de cada ingrediente.
- **Estoque.** Atualização das quantidades disponíveis de cada Produto Base e Ingrediente, refletindo o que efetivamente pode ser oferecido ao Cliente no totem e no aplicativo. É essa informação de estoque que impede, por exemplo, que um cliente monte um pedido com um ingrediente esgotado.

Essas operações de catálogo e estoque feitas pelo Atendente/Administrador alimentam diretamente os dados que o Cliente consome nos passos 2 e 3 do fluxo principal descrito anteriormente: a lista de produtos disponíveis e o preço de cada ingrediente vêm dessa gestão interna. Por isso, embora os dois perfis usem canais e telas distintas, suas interfaces não são independentes – são duas pontas do mesmo ciclo de dados do sistema.

Capítulo 5

Especificação e Estratégia de Testes

O Sistema Unespão concentra praticamente toda a lógica de negócio na API Unespão (.NET 8 / ASP.NET Core), enquanto o Web App/Totem funciona como uma camada de apresentação relativamente fina, que consome essa API via HTTPS/JSON. Por isso a estratégia de testes da equipe se concentra no backend: é ali que vivem as regras que decidem se um pedido pode ser fechado, se há insumo suficiente no estoque e como um item personalizado tem seu preço calculado. Este capítulo descreve os níveis de teste adotados, os critérios usados para selecionar o que testar e as ferramentas do ecossistema .NET que sustentam essa estratégia.

Um ponto de partida importante é a própria arquitetura em camadas descrita no capítulo de Arquitetura (Controllers → Services → Repositories/External Adapters → Domain Models, com fluxo de dependência unidirecional). Como os Services dependem de abstrações – `IPedidoRepository`, `IEstoqueRepository`, `IGatewayPagamentoAdapter`, entre outras – e não de classes concretas, a própria organização do código já favorece testes isolados: basta substituir a implementação real por um dublê de teste sem tocar em nenhuma linha da classe testada.

5.1 Planejamento e níveis de teste

Adotamos três níveis de teste, cada um respondendo a uma pergunta diferente sobre o sistema.

Os **testes de unidade** verificam se uma classe de negócio se comporta corretamente quando isolada de tudo à sua volta. O alvo principal aqui são os Services (`PedidoService`, `EstoqueService`, `ProdutoService` etc.) e os Domain Models com lógica própria, como `ItemPersonalizado`. A pergunta que esse nível responde é: esta unidade de código faz o que deveria, dado um conjunto controlado de entradas?

Já os **testes de integração** verificam se as peças que se comunicam entre si – no nosso caso, principalmente a API Unespão e o banco PostgreSQL via Entity Framework Core/Npgsql – realmente conversam corretamente quando combinadas. Aqui a pergunta muda: não é mais “a lógica está certa isoladamente”, e sim se o Repository gera o SQL certo, se o mapeamento do EF Core está coerente com o schema e se a operação persiste e recupera os dados como esperado.

Não chegamos a formalizar um nível de teste de sistema (ponta a ponta, via totem/app) neste trabalho. A ausência de protótipos de interface implementados (ver capítulo de Interface do Usuário) torna esse nível prematuro no momento, e registramos isso como um limite consciente do escopo, não como uma lacuna esquecida.

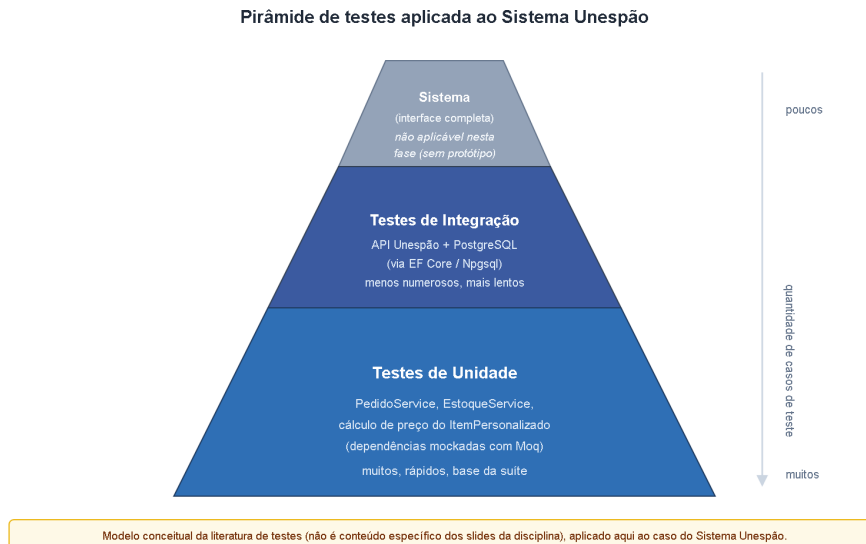


Figura 5.1: A pirâmide de testes é um modelo conceitual conhecido da literatura de teste de software (não um conteúdo específico dos slides da disciplina), usado aqui apenas para situar visualmente os dois níveis que de fato adotamos – muitos testes de unidade rápidos na base, menos testes de integração acima – e por que o nível de sistema, no topo, fica de fora nesta fase.

A prioridade de testes segue a mesma lógica dos objetivos de qualidade definidos na Introdução: como “Confiabilidade da informação de estoque” e “Segurança dos dados e transações do cliente” foram marcados como prioridade Alta, **EstoqueService** e o fluxo de pagamento (**PedidoService** + **IGatewayPagamentoAdapter**) recebem mais atenção de teste do que, por exemplo, **SugestaoPersonalizadaService**, cujo objetivo de qualidade associado (“Relevância das sugestões personalizadas”) tem prioridade Média.

5.2 Testes de unidade

A ideia central desta seção é a mesma trabalhada no exercício de testes de unidade da disciplina (**ProcessadorVendas** com e sem Mockito), só que transposta para o ecossistema real do projeto: em vez de JUnit e Mockito sobre um **ProcessadorVendas** de exemplo, usamos **xUnit** como framework de teste e **Moq** como biblioteca de mocks sobre as classes reais do Sistema Unespão, escritas em C#.

PedidoService com IPedidoRepository mockado

O **PedidoService** é o candidato mais natural para essa técnica, porque o próprio documento de arquitetura já cita esse par como exemplo de LSP: **PedidoRepository** (PostgreSQL real) e **MockPedidoRepository**/dublê de teste devem ser intercambiáveis do ponto de vista do **PedidoService**. Um teste sem mock, usando uma implementação em memória do repositório, ficaria assim:

```

1 public class PedidoServiceSemMockTests
2 {
3     [Fact]
4     public void FinalizarPedido_ComEstoqueSuficiente_RetornaSucesso()
5     {
6         var repositorioEmMemoria = new PedidoRepositoryEmMemoria();
7         var estoqueService = new EstoqueServiceEmMemoria();
8         var gatewayFake = new GatewayPagamentoSempreAprova();
9
10        var service = new PedidoService(repositorioEmMemoria, estoqueService, gatewayFake);

```

```

11         var resultado = service.FinalizarPedido(pedidoId: 1);
12
13         Assert.Equal(StatusPedido.Confirmado, resultado.Status);
14     }
15 }
16

```

Listing 5.1: Teste sem mock, com repositório e gateway em memória.

Essa abordagem funciona, mas exige manter classes auxiliares “de mentira” (`PedidoRepositoryEmMemoria`, `GatewayPagamentoSempreAprova`) só para viabilizar o teste, e qualquer mudança de comportamento nelas pode mascarar um defeito real. A abordagem com Moq resolve isso criando o duplê dinamicamente, a partir da própria interface:

```

1 public class PedidoServiceComMockTests
2 {
3     private readonly Mock<IPedidoRepository> _repositorioMock;
4     private readonly Mock<IEstoqueService> _estoqueMock;
5     private readonly Mock<IGatewayPagamentoAdapter> _gatewayMock;
6     private readonly PedidoService _service;
7
8     public PedidoServiceComMockTests()
9     {
10         _repositorioMock = new Mock<IPedidoRepository>();
11         _estoqueMock = new Mock<IEstoqueService>();
12         _gatewayMock = new Mock<IGatewayPagamentoAdapter>();
13         _service = new PedidoService(_repositorioMock.Object, _estoqueMock.Object,
14                                     _gatewayMock.Object);
15     }
16
17     [Fact]
18     public void FinalizarPedido_PagamentoAprovado_AtualizaStatusParaConfirmado()
19     {
20         var pedido = new Pedido { Id = 1, Status = StatusPedido.Aberto, ValorTotal = 35.90m };
21         _repositorioMock.Setup(r => r.GetById(1)).Returns(pedido);
22         _estoqueMock.Setup(e => e.HaSaldoSuficiente(pedido)).Returns(true);
23         _gatewayMock.Setup(g => g.Cobrar(It.IsAny<string>(), pedido.ValorTotal)).Returns(true);
24
25         var resultado = _service.FinalizarPedido(1);
26
27         Assert.Equal(StatusPedido.Confirmado, resultado.Status);
28         _repositorioMock.Verify(r => r.Update(It.Is<Pedido>(p => p.Status == StatusPedido.Confirmado)), Times.Once);
29     }
30
31     [Fact]
32     public void FinalizarPedido_PagamentoRecusado_MantemPedidoAbertoENaoDaBaixaNoEstoque()
33     {
34         var pedido = new Pedido { Id = 2, Status = StatusPedido.Aberto, ValorTotal = 20.00m };
35         _repositorioMock.Setup(r => r.GetById(2)).Returns(pedido);
36         _estoqueMock.Setup(e => e.HaSaldoSuficiente(pedido)).Returns(true);
37         _gatewayMock.Setup(g => g.Cobrar(It.IsAny<string>(), pedido.ValorTotal)).Returns(false);
38
39         var resultado = _service.FinalizarPedido(2);
40
41         Assert.Equal(StatusPedido.PagamentoRecusado, resultado.Status);
42         _estoqueMock.Verify(e => e.DarBaixa(It.IsAny<Pedido>()), Times.Never);
43     }
44 }

```

Listing 5.2: Testes com Moq, cobrindo pagamento aprovado e pagamento recusado.

O segundo teste é particularmente útil porque exercita um cenário que seria trabalhoso de forçar com um gateway real (uma recusa de pagamento) e verifica uma consequência que importa para a confiabilidade do estoque: se o pagamento falha, a baixa de insumos não pode acontecer. Isolar o `PedidoService` do gateway de pagamento real também evita que a suíte de testes dependa de rede, de credenciais ou da disponibilidade de um serviço externo, o que tornaria os testes lentos e instáveis.

EstoqueService: baixa de insumos

O `EstoqueService` concentra uma regra sensível: dar baixa nos insumos (produtos base e ingredientes) associados aos itens de um pedido, sem deixar o saldo ficar negativo. Aqui a atenção recai sobre os casos de borda: saldo exato, saldo insuficiente, item com quantidade zero.

```
1 [Fact]
2 public void DarBaixa_ComSaldoExato_ZeraEstoqueSemErro()
3 {
4     var estoqueMock = new Mock<IEstoqueRepository>();
5     estoqueMock.Setup(r => r.GetByIngredienteId(10)).Returns(new Estoque {
6         QuantidadeDisponivel = 2 });
7
8     var service = new EstoqueService(estoqueMock.Object);
9     service.DarBaixa(ingredienteId: 10, quantidade: 2);
10
11     estoqueMock.Verify(r => r.Update(It.Is<Estoque>(e => e.QuantidadeDisponivel == 0)), Times.
12         Once);
13 }
14 [Fact]
15 public void DarBaixa_ComSaldoInsuficiente_LancaExcecaoENaoAtualizaEstoque()
16 {
17     var estoqueMock = new Mock<IEstoqueRepository>();
18     estoqueMock.Setup(r => r.GetByIngredienteId(10)).Returns(new Estoque {
19         QuantidadeDisponivel = 1 });
20
21     var service = new EstoqueService(estoqueMock.Object);
22
23     Assert.Throws<EstoqueInsuficienteException>(() => service.DarBaixa(ingredienteId: 10,
24         quantidade: 2));
25     estoqueMock.Verify(r => r.Update(It.IsAny<Estoque>()), Times.Never);
26 }
```

Listing 5.3: Testes de `EstoqueService` cobrindo saldo exato e saldo insuficiente.

Na prática, o segundo caso é o mais importante dos dois: ele garante que uma tentativa de baixa acima do saldo disponível não corrompe silenciosamente o estoque, algo diretamente ligado ao objetivo de qualidade de confiabilidade da informação de estoque discutido na Introdução.

Cálculo de preço de um `ItemPersonalizado`

Diferente dos exemplos anteriores, o cálculo de preço de um `ItemPersonalizado` é um caso de teste de unidade sem qualquer dependência externa a isolar: é lógica pura sobre os dados do próprio objeto (preço base do produto mais a soma dos preços adicionais dos ingredientes selecionados). Isso o torna um bom exemplo de teste “sem mock”, no espírito do primeiro teste do exercício de referência da disciplina:

```
1 public class ItemPersonalizadoTests
2 {
3     [Fact]
4     public void CalcularPrecoTotal_ComIngredientesAdicionais_SomaPrecoBaseEAdicionais()
5     {
6         var produtoBase = new ProdutoBase { Nome = "Pao Frances", PrecoBase = 8.00m };
7         var ingredientes = new List<Ingrediente>
8         {
9             new Ingrediente { Nome = "Queijo extra", PrecoAdicional = 3.50m },
10            new Ingrediente { Nome = "Bacon", PrecoAdicional = 4.00m }
11        };
12
13        var item = new ItemPersonalizado(produtoBase, ingredientes);
14
15        Assert.Equal(15.50m, item.PrecoTotal);
16    }
17
18    [Fact]
19    public void CalcularPrecoTotal_SemIngredientesAdicionais_IgualAoPrecoBase()
20    {
21        var produtoBase = new ProdutoBase { Nome = "Pao Frances", PrecoBase = 8.00m };
22        var item = new ItemPersonalizado(produtoBase, new List<Ingrediente>());
23    }
24 }
```

```

23     Assert.Equal(8.00m, item.PrecoTotal);
24 }
25 }
26 }

```

Listing 5.4: Teste puro (sem mock) do cálculo de preço de um item personalizado.

Critério de seleção das unidades testadas

Não pretendemos cobrir cada classe do sistema com o mesmo nível de detalhe, e o próprio template não pede isso. Priorizamos Services que carregam regra de negócio com impacto financeiro ou de integridade de dados (`PedidoService`, `EstoqueService`) e modelos de domínio com lógica não trivial (`ItemPersonalizado`). Classes essencialmente wrappers finos, como boa parte dos Controllers, que segundo a documentação de arquitetura “não contêm regras de negócio”, recebem menos atenção em teste de unidade e acabam cobertas, na prática, pelos testes de integração descritos a seguir.

5.3 Testes de integração

Os testes de integração do Unespão têm como alvo principal a comunicação entre a API Unespão e o banco PostgreSQL, via Entity Framework Core e o provider Npgsql, a mesma dupla usada em produção, conforme descrito na visão de containers da arquitetura. A ideia é validar o que um teste de unidade com repositório mockado não consegue: se o mapeamento das entidades (`Pedido`, `ItemPersonalizado`, `Estoque` etc.) está correto, se uma consulta EF Core realmente traz os dados esperados e se uma transação que abrange múltiplas tabelas (por exemplo, criar um pedido e dar baixa no estoque na mesma operação) se comporta de forma consistente.

Optamos por uma estratégia próxima do que a literatura chama de integração *bottom-up*: primeiro validamos isoladamente cada Repository contra um banco de testes real (não mockado), e só depois testamos a composição completa via Controllers, exercitando a pilha inteira (Controller → Service → Repository → banco). Faz sentido nesse projeto porque os Repositories são a camada mais próxima da tecnologia externa (PostgreSQL) e mais provável de esconder problemas que um mock nunca revelaria, como um mapeamento incorreto de coluna ou uma constraint de chave estrangeira violada.

Na prática, esses testes usam um banco PostgreSQL de teste dedicado – subindo, por exemplo, via um container Docker específico para a suíte de testes, para não interferir no banco de desenvolvimento –, populado com uma massa de dados mínima antes de cada execução e limpo ao final dela, de forma que os testes não dependam da ordem de execução nem deixem resíduo entre si.

```

1 public class PedidoRepositoryIntegrationTests : IClassFixture<PostgresTestFixture>
2 {
3     private readonly UnespaoDbContext _context;
4
5     public PedidoRepositoryIntegrationTests(PostgresTestFixture fixture)
6     {
7         _context = fixture.CreateContext();
8     }
9
10    [Fact]
11    public async Task Add_PersisteEBuscaPedidoComItens()
12    {
13        var repositorio = new PedidoRepository(_context);
14        var pedido = new Pedido { ClienteId = 1, Status = StatusPedido.Aberto, ValorTotal =
15            12.50m };
16
17        await repositorio.AddAsync(pedido);
18        var pedidoRecuperado = await repositorio.GetByIdAsync(pedido.Id);

```

```

19     Assert.NotNull(pedidoRecuperado);
20     Assert.Equal(StatusPedido.Aberto, pedidoRecuperado.Status);
21 }
22 }

```

Listing 5.5: Teste de integração de `PedidoRepository` contra um banco PostgreSQL de teste.

Quanto à regressão, o critério adotado é simples: toda a suíte de testes de unidade e de integração é executada localmente antes de abrir um pull request, e novamente antes de qualquer merge no `main`. Como o repositório já opera com pull requests revisados antes da integração – prática visível no próprio histórico de commits do projeto –, o teste de regressão se encaixa naturalmente nesse fluxo: um PR que quebra um teste existente evidencia isso antes da revisão ser concluída, e a expectativa da equipe é não mesclar código com testes falhando.

5.4 Critérios de conclusão e cobertura

Decidimos não adotar uma meta numérica de cobertura (por exemplo, “mínimo de 80% de linhas/branches cobertas”) como critério de conclusão dos testes. Essa escolha está em linha com a decisão já tomada, no capítulo de Introdução, de não especificar RNFs quantificados como desempenho de pico ou portabilidade neste trabalho acadêmico. Caso o template de entrega do curso traga um campo obrigatório de “percentual de cobertura alvo”, esse campo deve ser preenchido como “(A PREENCHER PELO GRUPO)” em vez de se inventar um número só para completar a seção.

O critério de conclusão adotado é qualitativo e segue a mesma priorização Alta/Média usada na tabela de objetivos de qualidade da Introdução. Para funcionalidades de prioridade Alta – usabilidade, confiabilidade da informação de estoque e segurança de dados/transações, o que na prática cobre `EstoqueService`, autenticação, pagamento e o CRUD de catálogo/estoque do Atendente/Administrador, além da edição ou cancelamento de um item antes da confirmação do pagamento – exigimos ao menos um teste de unidade cobrindo o caminho de sucesso e um cobrindo o principal caminho de falha ou exceção de cada método público relevante (saldo insuficiente, pagamento recusado, valor inválido), com uso de mock via Moq para isolar dependências externas como o gateway de pagamento e os repositórios, seguindo o mesmo princípio de DIP já adotado no capítulo de Arquitetura. Para funcionalidades de prioridade Média, como as sugestões personalizadas, basta um teste de unidade cobrindo o caminho de sucesso.

Esse critério nos parece mais defensável do que perseguir um número de cobertura isolado: um teste que apenas exercita a linha de código sem testar de fato as condições de contorno passa despercebido em métricas de cobertura de linha, mas não reduz o risco real de defeito. Num projeto acadêmico de escopo definido – não um produto em produção com SLA – um critério baseado em risco e prioridade de requisito tende a ser mais rastreável do que uma meta percentual arbitrária, que muitas vezes acaba incentivando testes de baixo valor só para “bater número”.

Isso não significa abrir mão de medir cobertura. A cobertura de linha e de branch, obtida via Coverlet, continua sendo acompanhada como métrica de diagnóstico, útil sobretudo para identificar Services críticos que ficaram sem nenhum teste, o que seria um sinal de alarme independentemente de qualquer meta numérica. O que fica descartado é usá-la como gate de aprovação com limiar obrigatório.

5.5 Automação e ferramentas

A stack de testes do backend .NET 8/ASP.NET Core do Sistema Unespão é composta por:

- **xUnit** como framework de teste unitário e de integração, por ser o padrão de facto para projetos .NET modernos e por sua boa integração com o `dotnet test` e com o Visual Studio/VS Code;
- **Moq** como biblioteca de mocks, usada para isolar Services de suas dependências (`IPedidoRepository`, `IEstoqueService`, `IGatewayPagamentoAdapter`, `IGoogleAuthAdapter` etc.) nos testes de unidade, no mesmo espírito do Mockito no exercício de referência da disciplina, porém aplicado às interfaces reais do projeto;
- **Coverlet** como ferramenta de medição de cobertura de código, integrado ao `dotnet test` via `dotnet test --collect:"XPlat Code Coverage"`, gerando relatórios que podem ser convertidos para HTML (por exemplo, via `ReportGenerator`) e inspecionados da mesma forma que o relatório do JaCoCo no exercício em Java;
- um **banco PostgreSQL de teste**, isolado do banco de desenvolvimento/produção, para os testes de integração que envolvem EF Core/Npgsql.

Essa troca de ferramentas em relação ao material de referência é deliberada: o exercício de testes de unidade da disciplina usa Java, JUnit, Mockito e JaCoCo, mas o backend real do Sistema Unespão é .NET 8/ASP.NET Core em C#. Documentar as ferramentas Java descreveria uma stack que o projeto não usa; por isso a equipe optou por reproduzir a mesma forma de trabalho – teste com e sem dublê, medição de cobertura – dentro do ecossistema .NET efetivamente empregado no projeto.

Sobre integração contínua: não há, até o momento, um pipeline de CI/CD configurado no repositório para rodar essa suíte automaticamente a cada push ou pull request. O que existe hoje é o uso de pull requests com revisão antes do merge no `main`, como mostra o histórico de commits do projeto. A execução dos testes (`dotnet test`) e a geração do relatório de cobertura, portanto, ainda são feitas manualmente pelos desenvolvedores antes de abrir ou aprovar um PR. Fica registrado como recomendação de boas práticas, e não como algo já implementado, configurar um workflow de CI (por exemplo, GitHub Actions) que rode `dotnet test` com coleta de cobertura automaticamente a cada pull request, complementando a revisão humana já praticada nos PRs com uma verificação automática antes do merge.

Capítulo 6

Gestão de Configuração e Manutenção

Este capítulo descreve como os artefatos produzidos ao longo do projeto do Sistema Unespão são versionados e mantidos sob controle, e como uma alteração feita em qualquer ponto do sistema pode ser acompanhada até seu efeito nos demais artefatos relacionados. O que está descrito aqui é o que a equipe efetivamente faz no repositório do projeto, sem inventar um processo mais formal do que o que está em uso.

6.1 Repositório e itens de configuração

O projeto é versionado em um repositório Git hospedado no GitHub, com a branch `main` funcionando como ponto de referência para a versão consolidada do trabalho. O README atual descreve o repositório de forma sucinta como “nosso repositório para servir de KB do nosso projeto”, frase que já dá uma ideia de como o Git ali serve tanto ao código quanto aos artefatos de documentação da disciplina.

Os principais itens de configuração controlados no repositório são o código-fonte da API back-end, desenvolvida em .NET 8/ASP.NET Core, responsável pela lógica de negócio, pelos endpoints consumidos pelo front-end e pela integração com o gateway de pagamento e demais adaptadores externos; o código-fonte do front-end, um SPA que implementa as telas de totem e de aplicativo do cliente, no framework definido no capítulo de Arquitetura para o projeto; o código dos testes automatizados descritos no capítulo de Estratégia de Testes, versionado junto do código de produção que ele cobre; as migrations do banco de dados, geradas via Entity Framework Core, que descrevem de forma incremental a evolução do esquema do PostgreSQL usado pelo sistema; a documentação do projeto, incluindo os arquivos fonte em LaTeX (`.tex`) deste documento e arquivos de apoio em Markdown (`.md`), como o próprio README; os diagramas C4 (Contexto, Contêineres e, quando aplicável, Componentes) usados para descrever a arquitetura do sistema nos capítulos anteriores; e o arquivo de referências bibliográficas (`refs.bib`), que consolida as fontes citadas ao longo da documentação. Esses últimos itens de documentação (fontes LaTeX, `refs.bib` e imagens de apoio) devem ser efetivamente adicionados e commitados no repositório antes da entrega, mantendo-os sob o mesmo controle de versão do código.

Um ponto de atenção sobre configuração sensível: a API depende de dados como a connection string do PostgreSQL e as credenciais do gateway de pagamento, que por natureza não deveriam ser commitados em texto plano junto do código-fonte. Nas fontes disponíveis do repositório não há evidência de uma separação formal já em uso para isso – não há, por exemplo, um `appsettings.Development.json` versionado com placeholders e um `appsettings.json` de produção fora do controle de versão, nem um `.gitignore` que explicita esse tipo de arquivo, nem uso de variáveis de ambiente documentado. Registramos isso como uma lacuna observável do estado atual do projeto, e não como algo já resolvido: à medida que a API for de fato

implementada, esses segredos precisam ficar fora do Git (via variáveis de ambiente, `appsettings.Development.json` ignorado por `.gitignore`, ou mecanismo equivalente), mantendo apenas arquivos de configuração com placeholders sob versionamento.

Tratamos as migrations como item de configuração porque, assim como o código, elas precisam estar sincronizadas com a versão da aplicação que as acompanha: uma API em determinada versão espera um esquema de banco em estado compatível, e um descompasso aí quebra a aplicação mesmo com o código correto. Todos esses itens convivem no mesmo repositório, o que ajuda a manter código e documentação evoluindo juntos. Quando uma decisão de arquitetura muda, por exemplo, o diagrama C4 correspondente e o texto do capítulo de Arquitetura costumam ser atualizados no mesmo conjunto de commits.

6.2 Gestão de dependências e alterações

As dependências externas são gerenciadas separadamente em cada camada da aplicação, seguindo o gerenciador de pacotes padrão de cada ecossistema. No back-end, as dependências do .NET – bibliotecas do próprio ASP.NET Core, driver do PostgreSQL, pacote do Entity Framework Core, cliente do gateway de pagamento, entre outras – são declaradas no arquivo de projeto `.csproj` e resolvidas pelo NuGet, que também registra as versões exatas utilizadas. No front-end, as dependências do SPA são declaradas no `package.json` e gerenciadas pelo npm, com as versões efetivamente instaladas fixadas no arquivo de lock correspondente.

O histórico do repositório mostra ainda que a equipe já adota Pull Requests como mecanismo de integração de mudanças à `main`, em vez de commits diretos. Isso aparece de forma explícita no próprio log do Git, com merges de PR registrados como eventos distintos do desenvolvimento (algo detalhado na seção de Rastreabilidade a seguir). Esse fluxo dá à equipe um ponto de revisão antes que qualquer mudança chegue à versão principal do projeto.

Quando uma alteração afeta mais de um artefato – por exemplo, uma mudança na regra de negócio do estoque que exige ajuste na API, uma migration nova e um trecho do capítulo de Arquitetura que descreve o `EstoqueService` (o serviço responsável por isolar a lógica de estoque, conforme o princípio de responsabilidade única discutido no capítulo de Arquitetura) – a expectativa é que essas mudanças entrem no mesmo Pull Request, de modo que a revisão avalie o conjunto coerente em vez de partes soltas. Isso ainda não está formalizado como uma convenção escrita da equipe (não há, por exemplo, um guia de contribuição ou template de PR no repositório), mas é a prática que o histórico de commits sugere até aqui.

6.3 Rastreabilidade

Um exemplo real e observável de rastreabilidade no projeto é o próprio histórico de integração via Pull Requests. O log do Git mostra, entre os commits mais recentes, os merges “Merge pull request #2 from 00davidсарago00/docs” e “Merge pull request #1 from 00davidсарago00/00davidсарago00-patch-1”. Cada um desses merges corresponde a um Pull Request identificável (#1 e #2) que passou por revisão no GitHub antes de ser incorporado à `main`, ou seja, dá para partir do identificador do PR e chegar ao conjunto de commits e arquivos que ele trouxe para o projeto.

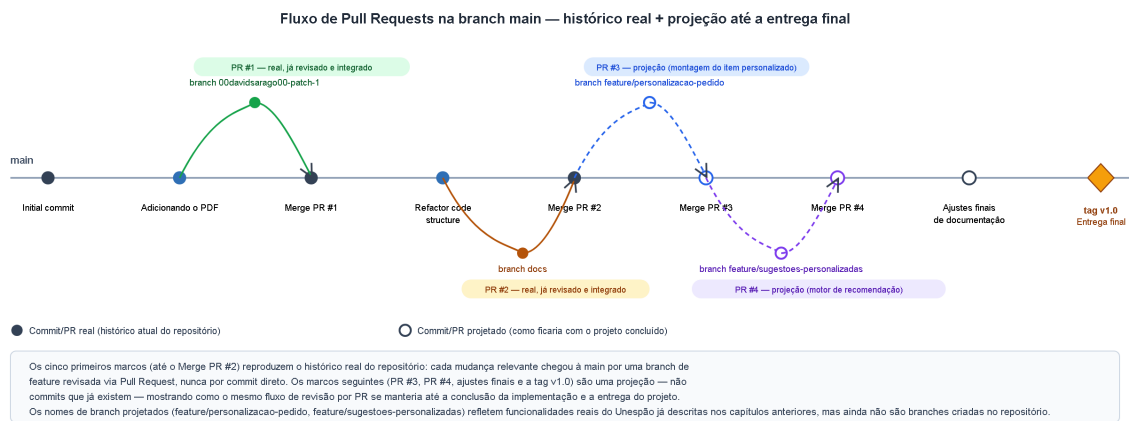


Figura 6.1: Linha do tempo da branch `main` combinando duas partes: os cinco primeiros marcos (até o Merge PR #2) reproduzem o histórico real do repositório, com as branches `00davidсарago00-patch-1` e `docs` integradas via PR #1 e PR #2; os marcos seguintes (PR #3, PR #4, ajustes finais e a tag `v1.0`), marcados com nós vazados e linhas tracejadas, são uma projeção ilustrativa de como o mesmo fluxo de revisão por Pull Request se manteria até a conclusão da implementação e a entrega do projeto — não commits ou branches que já existem no repositório.

Esse é o nível de rastreabilidade que a equipe consegue demonstrar hoje: da mudança (PR) para o artefato afetado (arquivos alterados no merge) e para o ponto exato do histórico em que ela entrou na branch principal (o commit de merge). Não existe ainda uma matriz formal ligando cada requisito ou issue a um PR e a um caso de teste específico, porque o projeto não usa um sistema de issues de forma sistemática. Ampliar esse rastreamento, vinculando issues do GitHub a PRs e a casos de teste, é um passo natural de evolução do processo, mas o que registramos aqui é apenas o que o histórico atual já evidencia.

6.4 *Build*, integração e entrega

Não há, nas fontes do repositório, confirmação de um pipeline de integração contínua configurado; não existe, por exemplo, workflow do GitHub Actions ou equivalente no projeto até o momento desta produção. O que existe de fato, como descrito nas seções anteriores, é a prática de submeter alterações via Pull Request e revisá-las antes do merge para a `main`, o que já funciona como uma barreira mínima de controle de qualidade antes da integração, mesmo sem automação.

Sem essa automação, o processo de build e execução do sistema é, hoje, manual: a API .NET seria compilada e executada localmente ou em ambiente de implantação a partir do código da branch principal, com as migrations do Entity Framework Core aplicadas ao banco PostgreSQL antes da subida da aplicação, enquanto o SPA seria compilado separadamente por seu próprio processo de build, no framework definido para o front-end, e servido de forma independente da API.

Como recomendação de boas práticas de Engenharia de Software para a evolução do projeto — e não como algo já implementado — sugerimos que a equipe adote futuramente um pipeline de integração contínua acionado a cada Pull Request, capaz de restaurar as dependências (NuGet e npm), compilar back-end e front-end, aplicar as migrations em um banco de teste e rodar a suíte de testes automatizados antes de permitir o merge. Isso transformaria em verificação automática o controle de qualidade que hoje depende só da revisão manual do PR, reduzindo o risco de uma alteração quebrar o build da `main` sem que ninguém perceba antes da entrega.

Controle de Versões do Documento

A tabela a seguir registra a evolução deste documento de projeto final em si, não do código-fonte do Sistema Unespão. Esse segue o fluxo de Git e Pull Requests já descrito no capítulo de Gestão de Configuração.

Tabela 6.1: Histórico de versões do documento.

Data	Autor(es)	Versão	Alteração
12/09/2026	Equipe Unespão	0.1	Versão inicial da estrutura do documento, com os capítulos organizados conforme o template do projeto final.
12/09/2026	Equipe Unespão	1.0	Versão consolidada para entrega, com revisão geral do conteúdo de todos os capítulos e ajustes finais de redação e formatação.

Referências bibliográficas

Patrick Roos. *The Ultimate Guide To Software Architecture Documentation*. workingsoftware.dev, 2023.

Gernot Starke and Peter Hruschka. *arc42*. arc42, 2024. URL <https://arc42.org>.